

Data Structure (week 4)

講師：陳俊豪

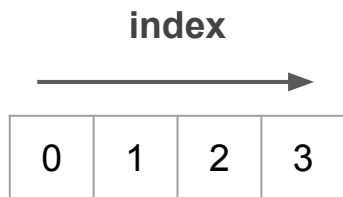
Python 套件介紹

Numpy 科學計算基礎

Numpy 資料型態

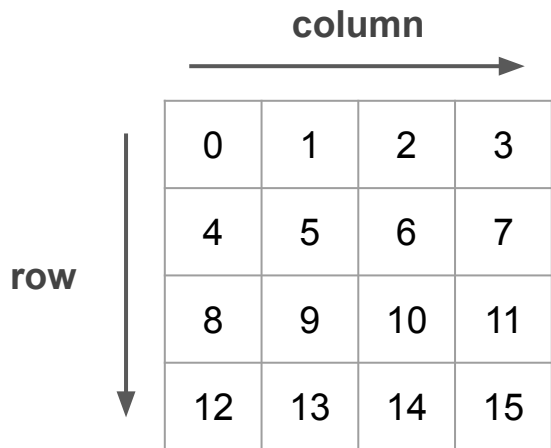
- Python 原本的四種變數型態有時不符合數學運算需求, Numpy 提供了矩陣型態。

一維向量 vector



Shape : (index,)

二維矩陣 matrix



Shape : (row, column)

多維陣列 array

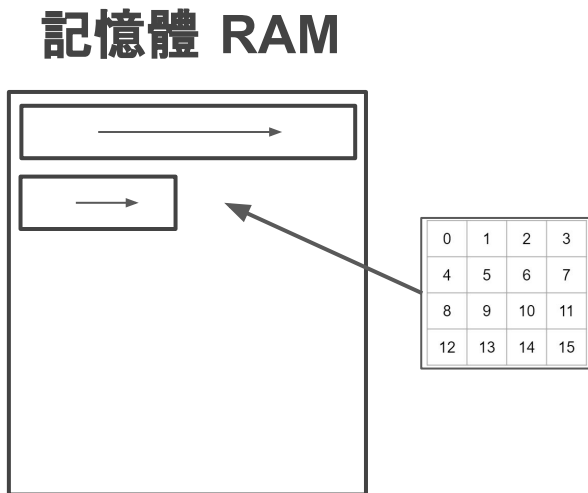
大家都是
ndarray



Shape : (a, b, c, d)

Numpy 資料型態

- Python 原本的四種變數型態有時不符合數學運算需求, Numpy 提供了矩陣型態。



Numpy

C++ based

真正的矩陣

```
import numpy as np
```

```
matrix = np.array([  
    [1, 1, 1],  
    [2, 2, 2],  
    [3, 3, 3]  
])
```

使用 import
在 python
中的第三方
套件引入。

Numpy 更多的 Slicing

- Numpy 做 Slicing 時是非常自由的, 詳情看以下。

矩陣 Matrix

	column →			
row ↓	0	1	2	3
	4	5	6	7
	8	9	10	11
	12	13	14	15

matrix[0, :] # 取 row

matrix[1:, 3] # 取 col

matrix[2:, :2] # 取區塊

多維陣列 **array**



Shape : (a, b, c, d)

Numpy 其他細節補充

- Numpy 的 array 只能用在數值上 (int, float), 其數值精度可以要求到很細。

32 精度 (同理 float)

```
arr = np.array([1, 2, 3], dtype='int32')
```

```
arr = np.array([1, 2, 3], dtype=np.int32)
```

64 精度 (同理 int)

```
arr = np.array([1, 2, 3], dtype='float64')
```

```
arr = np.array([1, 2, 3], dtype=np.float64)
```

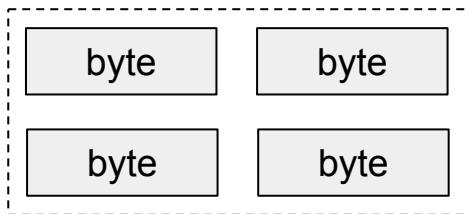
- 那什麼是精度, 在深度學習好像很常聽到?

Numpy 其他細節補充

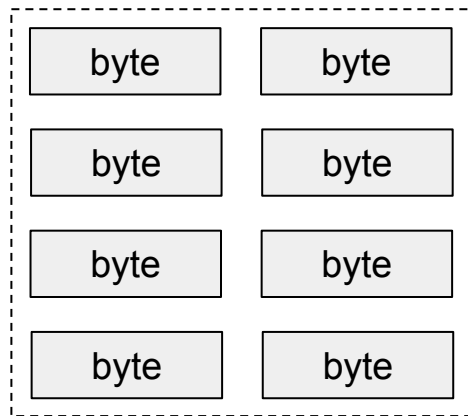


- 精度顧名思義代表要電腦去記一個數直到多精細。(ex. 1, 0.1234567, 1.000000017)
- 電腦可操作 (定址) 最小單位是 byte, 而 1 byte 又包含 8 bits

32 精度



64 精度



- 深度學習或其他科學運算上, 經常要求到很精準的成果 (ex. 微分一次差一點, 但微個 100w 次就天差地遠, 重點在**誤差擴大**)

Python 中變數使用多少 bytes ?

- 布林值 (bool) :
 - True : 28 bytes (1)
 - False : 24 bytes (0)
- 整數 (int) :
 - 大於 24 bytes 的動態記憶, 無上限
- 符點數 (float) :
 - **固定** 24 bytes (16 型別標籤 + 8 數值)
- 字串 (str) :
 - 空字串 "" : 41 ~ 49 bytes (不同版本)

Python 中變數使用多少 bytes ?

- 布林值 (bool) :
 - True : 28 bytes (1)
 - False : 24 bytes (0)
- 整數 (int) :
 - 大於 24 bytes 的動態記憶, 無上限
- 浮點數 (float) :
 - 固定 24 bytes (16 型別標籤 + 8 數值)
- 字串 (str) :
 - 空字串 "" : 41 ~ 49 bytes (不同版本)

無聊補充 :

型別標籤 16 = 8 (檢查是否觸發 Garbage Collection) + 8 (Type Pointer 是哪個 type)

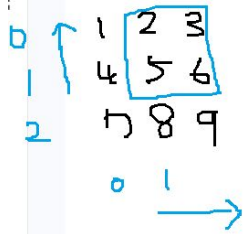
單個變數 與 64 精度 Numpy (8 bytes), 在底層幾乎是一樣的

但多個變數 就完全不一樣了 ! (List vs Array)

實務上 多使用 32 精度 (4 bytes), 以增加訓練、運算效率, 大多情況如此精準不是重點 (除非航天計算、物理碰撞模擬、誤差會放大的計算)

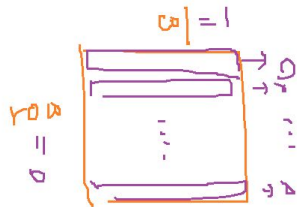
連續存在

In-class exercises 1

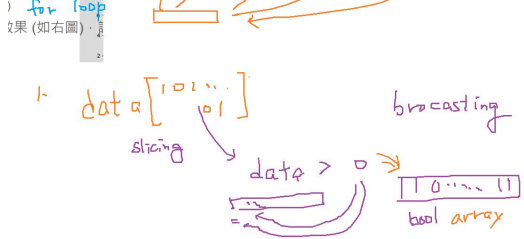


- 假設有一個 `arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])`。請問執行 `arr[2, 1:]` 會得到什麼結果？
- 假設有 numpy `arr1 = [10, 20, 30]` 與 `arr2 = [2, 2, 3]`。若要計算各元素相乘，哪種寫法最正確？
 - `ans = [x * y for x, y in zip(arr1, arr2)]`
 - `ans = np.array(list(map(lambda x, y: x * y, arr1, arr2)))`
 - `ans = arr1 * arr2`
- 假設有一個形狀為 (10000, 10000) 的二維陣列 `arr`。若要計算「每 row 的總和」該怎麼寫？

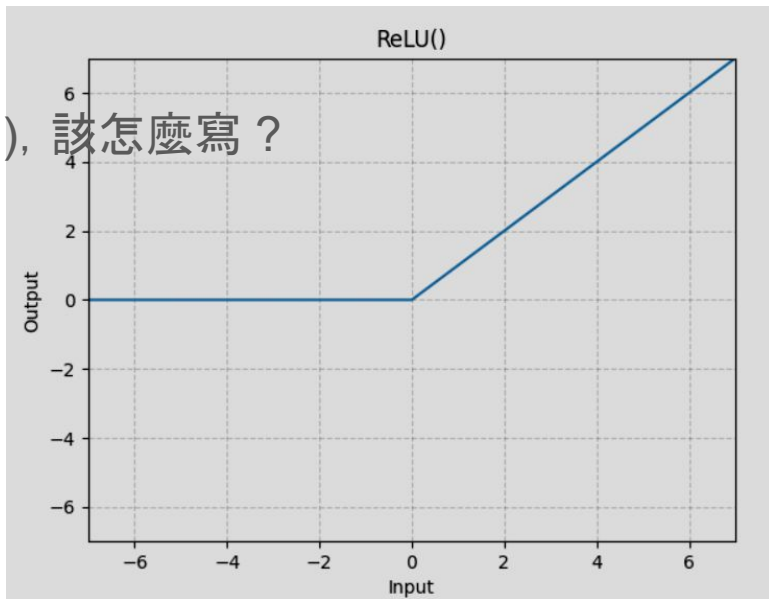
$\text{shape} = (10000, 10000) = \text{arr}$
 $\text{np.sum}(\text{arr}, \text{axis} = 1)$



In-class exercises 2



- 假設你有一個包含一百萬個隨機浮點數的一維陣列 `data`。現在你想把其中「大於 0 的數字」全部挑出來組成一個新的陣列。下列哪一種寫法執行速度最快？
 - `ans : data[data > 0]`
 - `ans : np.array([x for x in data if x > 0])`
- 假設需要實作一個 ReLU 函數的效果 (如右圖), 該怎麼寫？
 - (x) `np.array([0 if x < 0 else x for x in arr])`
 - `ans : arr[arr < 0] = 0`
 - `ans : np.maximum(arr, 0)`



In-class exercises 3

- 幽靈 np.nan :

- np.nan 是在資料分析時遇到 **缺失值資料** 來用來填補代表 **Not a Number** 的資訊, 但他有個反直覺的特性, 就是 `np.nan == np.nan` 會得到 False, 為什麼?
- 原因:
 - 因為 numpy 設計 nan 時嚴格遵守 IEEE 754 的浮點數國際標準: 任何「無效或未定義的數字」在數學上都不等於另一個「無效或未定義的數字」。直白來說: 純數學中 $0/0$ 和 $\infty - \infty$ 就是 undefined, 但 CS 中需要有錯誤代碼因此出現 NaN, 不過這就導致超多可能都會出現 NaN, 因此 IEEE 委員會訂定時強制規定為 False, 因為很難知道是什麼原因導致的 NaN ~
- Takeaway:
 - 千萬不要用 `arr == np.nan` 來找缺失值, 什麼都找不到!
 - 要使用 numpy 自己的 `np.isnan(arr)`

Array

陣列

Static Array (傳統陣列)

- Numpy 為了追求計算速度, 會把所有 bytes 都連在一起, 使得 CPU 能發揮「空間局部性」優勢, 才能海放 Python List
- Static Array
 - 發明自 1950-1960 年代
 - **優點 1**: 記憶體**連續**擺放, 大幅提升運算效率
 - **優點 2**: 取資料的過程 (slice, index) 也只需要 **$O(1)$**
 - (簡略版公式): 目標記憶體位址 = 起始位址 + 索引值
 - **Random Access** (隨機存取)
 - **缺點**: 固定大小, 沒有彈性
 - 這就是為什麼 numpy 沒有 append ?
 - **Insert** 與 **Delete** 時, 所有人都要 **shift**, 時間複雜度高達 **$O(N)$** (Q: insert 後元素不動還是 +1 ?)

時間複雜度 (Time Complexity)

- 什麼是時間複雜度？
 - 不是計算程式跑了「幾秒鐘」、「幾分鐘」、「幾小時」
 - 10 筆資料 vs 100 萬筆資料
 - 是評估：當資料量 (N) 越來越大時，程式執行的「步驟數量」會跟著變多少？
- 專用符號 Big O Notation (O) :
 - **O(1)** : 常數時間
 - 不管資料有多少 (ex. 10 筆 vs 100 萬筆), 程式執行的步驟數量「沒變」
 - 想一下剛剛例子, 10 筆 vs 100 萬筆以下程式執行的步驟數量各是多少？
 - 目標記憶體位址 = 起始位址 + 索引值
 - 目標記憶體位址 = 起始位址 + (索引值 * 8)

時間複雜度 (Time Complexity)

- 什麼是時間複雜度？

- 不是計算程式跑了「幾秒鐘」、「幾分鐘」、「幾小時」
 - 10 筆資料 vs 100 萬筆資料
- 是評估：當資料量 (N) 越來越大時，程式執行的「步驟數量」會跟著變多少？

- 專用符號 Big O Notation (O)：

- **O(1)**：常數時間
 - 不管資料有多少 (ex. 10 筆 vs 100 萬筆)，程式執行的步驟數量「沒變」
- **O(N)**：線性時間
 - 資料量增加，程式執行的步驟就跟著增加
 - ex. [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] loop vs double loop
- 還有很多種：ex. $O(2^N)$ 、 $O(N^2)$ 、 $O(N \log N)$... 等等

空間複雜度 (Space Complexity)

- 什麼是空間複雜度？
 - 不是指「程式 or 資料」佔了多大空間
 - 10 筆資料 vs 100 萬筆資料
 - 是評估：當資料量 (N) 越來越大時，程式執行的會需要增加多少「額外記憶體空間」？
- 專用符號 Big O Notation (O) :
 - **O(1)** : 常數空間
 - 不管資料有多少 (ex. 10 筆 vs 100 萬筆), 程式執行需要的額外空間「沒變」
 - 想一下剛剛例子，10 筆 vs 100 萬筆以下程式執行需要的額外空間各是多少？
 - 目標記憶體位址 = 起始位址 + 索引值
 - 目標記憶體位址 = 起始位址 + (索引值 * 8)

空間複雜度 (Space Complexity)

- 什麼是空間複雜度？
 - 不是指「程式 or 資料」佔了多大空間
 - 10 筆資料 vs 100 萬筆資料
 - 是評估：當資料量 (N) 越來越大時，程式執行的會需要增加多少「額外記憶體空間」？
- 專用符號 Big O Notation (O)：
 - $O(1)$ ：常數空間
 - 不管資料有多少 (ex. 10 筆 vs 100 萬筆)，程式執行需要的額外空間「沒變」
 - $O(N)$ ：線性空間
 - 資料量增加，程式執行的步驟就跟著增加
 - ex. [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] copy array
 - 還有很多種：ex. $O(2^N)$ 、 $O(N^2)$ 、 $O(N \log N)$... 等等



例子：程式碼都能執行，但速度差很多

- **情境**：從資料中查詢某位會員的消費紀錄 (100 筆 vs 1000 萬筆)

時間複雜度	程式邏輯	N = 100	N = 1000 萬
O(1)	Numpy	1 步	1 步
O(N)	List + Loop	100 步	1000 萬步
O(N^2)	Double Loop	10000 步	100 兆步

結論：

在 1000 萬筆資料下，一瞬間 (1 步) vs 無法達成任務 (100兆步)，就會是天與地的差別。這就是為什麼不能用一個 List 打天下，選擇**正確的資料結構**，能讓百萬級的運算從幾小時變成一瞬間！

用空間換取時間 (Space-Time Tradeoff)

- 為什麼大家都在追求「時間複雜度」，而不是「空間複雜度」？
 -

用空間換取時間 (Space-Time Tradeoff)

- 為什麼大家都在追求「時間複雜度」，而不是「空間複雜度」？
 - **時間就是金錢**！能用 $O(1)$ 解決，絕不用 $O(N)$
 - **儲存成本大幅下降**，遠低於 GPU/CPU
 - **結論**：在多數商業環境中，寧願犧牲空間，也要換取最快的速度！
- 記憶體控制還是很重要：
 - RAM 與 **VRAM** 仍然很貴 (下一頁補充)
 - OOM 錯誤 (Out of Memory) 直接當機，跑了 10 天的結果瞬間歸零

(補充) D-RAM vs V-RAM

- D-RAM (系統記憶體)

- 由 CPU 調用, 便宜、容量大、擴充很容易
 - 機器學習、增強式學習
- D 代表 Dynamic, 是因為電容需要動態充電, 不是 DDR4、DDR5 ...
- 那有 Static - RAM 嗎 ... ?

- V-RAM (顯示卡記憶體)

- 由 GPU 調用, 專門處理巨量且重複的平行運算, 速度極快, 但極度昂貴且容量小
 - 深度學習、AI 大語言模型
- V 代表 Video, 最早是為了影片、遊戲而生

Static Array (傳統陣列)

- Numpy 為了追求計算速度，會把所有 bytes 都連在一起，使得 CPU 能發揮「**空間局部性**」優勢，才能海放 Python List
 - 當 CPU 讀取 Array[0] 時，因為記憶體是連續的，CPU 會猜你之後也會看 ex. Array[2]、Array[3:7] 等，因此附近的 Array 會把打包塞到 CPU Cache (S-RAM) 裡！
 - (具體是「一排」，像是 Array[0] ~ Array[7]，又稱 Cache Line)
- 反觀 List 把資訊散落在 D-RAM 的各個角落，每一筆都要重找 ...
 - CPU Cache 在這也發揮不上任何作用，差太遠了！
- **結論：**
 - Array 不只運算效率高 $O(1)$ ，還完美配合了現代 CPU 的硬體設計

Static Array (傳統陣列)

- Numpy 為了追求計算速度, 會把所有 bytes 都連在一起, 使得 CPU 能發揮「空間局部性」優勢, 才能海放 Python List
- Static Array
 - 發明自 1950-1960 年代
 - **優點 1**: 記憶體連續擺放, 大幅提升運算效率
 - **優點 2**: 取資料的過程 (slice, index) 也只需要 **$O(1)$**
 - (簡略版公式): 目標記憶體位址 = 起始位址 + 索引值
 - **Random Access** (隨機存取)
 - **缺點**: 固定大小, 沒有彈性
 - 這就是為什麼 numpy 沒有 append ?
 - **Insert** 與 **Delete** 時, 所有人都要 **shift**, 時間複雜度高達 **$O(N)$**

Dynamic Array (動態陣列)

- Static Array 的缺點 (Numpy) :
 - 固定大小，沒有彈性！
 - Insert 與 Delete 時，所有人都要 shift，時間複雜度高達 $O(N)$
- Dynamic Array : 兩層架構，有彈性，但還是會搬家
 - Step 1. 預留空間：一開始先跟作業系統要一塊連續的記憶體空間 (ex. 4 個位置)
 - Step 2. 正常使用：前 4 個資料直接放進去，瞬間完成 $O(1)$
 - Step 3. 需求過多：當需求超過空間時 (ex. 第 5 個)，去找一塊新的「更大」的新連續空間
 - Step 4. 無情搬遷：將舊資料全部複製到新位置 (耗時 $O(N)$)，最後把舊記憶體註銷。
- 以上是 C/C++ 正常的運作過程，但是 ...

Dynamic Array (動態陣列)

- Static Array 的缺點 (Numpy) :

- 固定大小, 沒有彈性 !
- Insert 與 Delete 時, 所有人都要 shift, 時間複雜度高達 $O(N)$

- Dynamic Array : 兩層架構, 會搬家 > 有彈性

- Step 1. 預留空間 : 一開始先跟作業系統要一塊連續的記憶體空間 (ex. 4 個位置)
- Step 2. 正常使用 : 前 4 個資料直接放進去, 瞬間完成 $O(1)$
- Step 3. 需求過多 : 當需求超過空間時 (ex. 第 5 個), 去找一塊新的「更大」的新連續空間
- Step 4. 無情搬遷 : 將舊資料全部複製到新位置 (耗時 $O(N)$), 最後把舊記憶體註銷。

- 以上是 C/C++ 正常的運作過程, 但是 ...

如果是 **append**, 那其實分攤後是 $O(1)$!!!

假設執行 N 次 **append** 且滿 2 的倍數要搬家 ex. 2、4、8 ...

第 1 次搬家 : 去找 2 個記憶體位置, 搬了 1 筆資料

第 2 次搬家 : 去找 4 個記憶體位置, 搬了 2 筆資料

第 3 次搬家 : 去找 8 個記憶體位置, 搬了 4 筆資料

依此類推會是 $1 + 2 + 4 + \dots + N/2 = N - 1$ 必小於 N

總花費時間 = (一般放入 N) + (搬家複製 $< N$) = 小於 $2N$

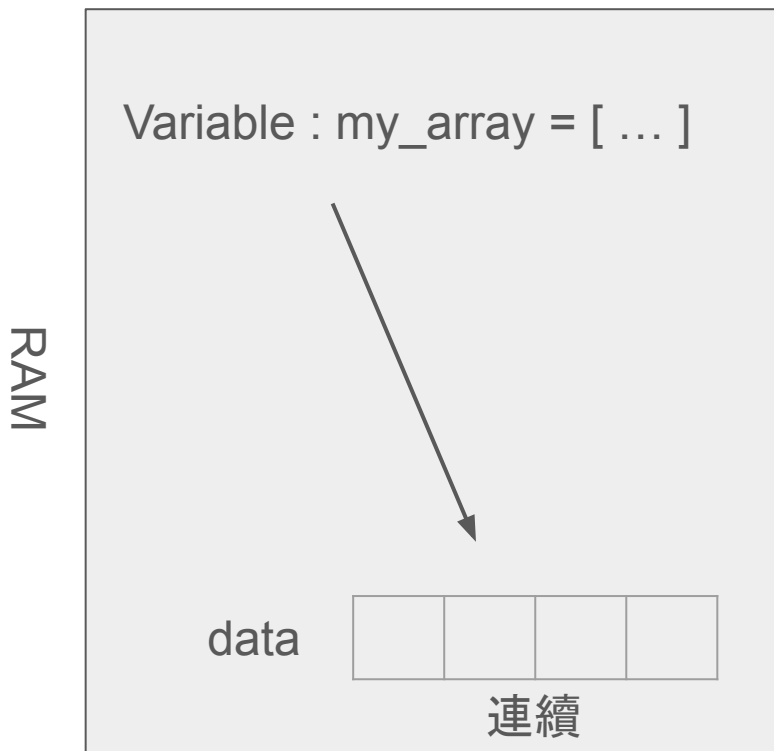
但如果是 **insert**, 那仍舊是 $O(N)$ 無法解決,
因為每次都會把後面的元素全部往右移

Dynamic Array (動態陣列)

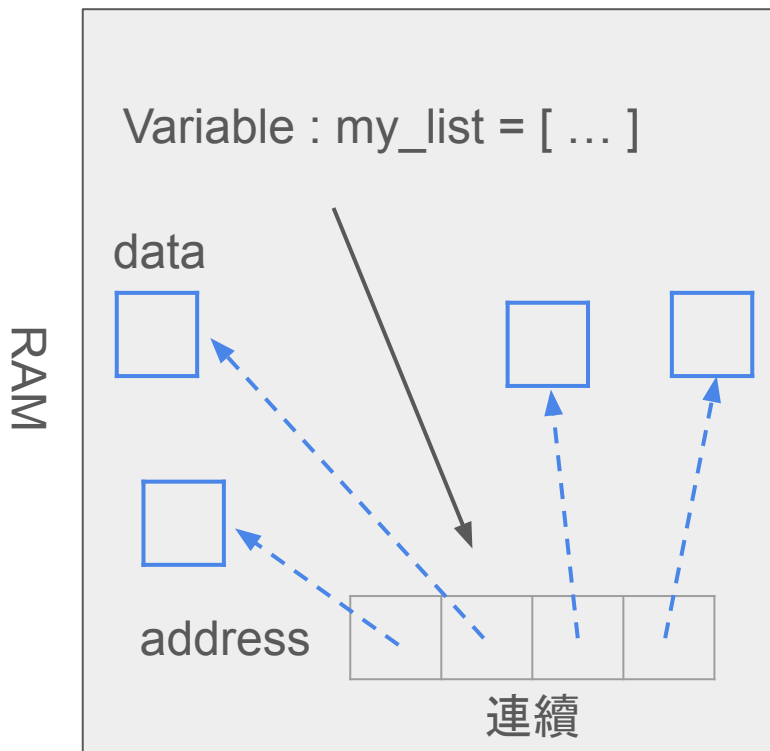
- Python 內的 Dynamic Array 採用的是「**三層架構**」
 - Step 1. 預留空間：一開始先跟作業系統要一塊連續的記憶體空間 (ex. 4 個位置), 但是拿來放「**數值的地址**」, 沒錯! 那些地址是分散的!
 - Step 2. 正常使用：前 4 個資料, 先把實體資料隨機丟在記憶體某處, 接著直接把這些資料的「地址」放進去連續的記憶體空間, 瞬間完成 $O(1)$
 - Step 3. 需求過多：當 4 個地址位子寫滿 (ex. 第 5 個要進來), 去找一塊新的「更大」的連續空間來當新通訊錄
 - Step 4. 無情搬遷：將舊的「地址」全部複製到新通訊錄 (耗時 $O(N)$)。注意：真正的資料留在原地, 完全沒有移動!

比較示意圖：Python

Static Array (Numpy)



Dynamic Array (List)



In-class exercises 1

- 在 Python 中，如果你有一個包含 100 萬筆客戶資料的 list。現在老闆要求你在名單的「最前面 (index=0 的位置)」強行插入一筆客戶新資料，也就是執行 `list.insert(0, new_data)`。請問這個動作的時間複雜度是多少？
 - Ans = $O(N)$
 - 理由：
 - 為了騰出最前面的空間，原本 100 萬筆資料的地址全部都必須往後 shift 一格。且資料越大花的時間就越多，因此為 $O(N)$

In-class exercises 2

- Dynamic Array (例如 Python List) 解決了傳統陣列「大小固定」的痛點。當預留的空間客滿，但又必須 append 新資料時，底層的「搬家機制」是如何運作的？
 - Ans = 去找一個更大的記憶體位置把舊的地址都複製過去，再註銷舊記憶體
 - 理由：
 - 注意雖然搬家耗時 $O(N)$ ，但平攤下來仍有 $O(1)$ 的極佳效能

In-class exercises 3

- 前面我們討論過 append 和 insert 在 Static / Dynamic Array 的問題，那如果是 delete 呢，對 Static / Dynamic Array 的時間複雜度是多少？
 - Hint : 分別討論 pop() 與 remove()
 - Ans : pop 為 $O(1)$
 - 不管是 Static 還是 Dynamic, 只要是拿走最後一個, 都不會產生「空洞」, 後面也沒人需要往前遞補
 - Ans : pop(N) 為 $O(N)$
 - 會產生一個空洞, 因為陣列必須「連續」, 所以後面的所有人都要往前推一格來填補

補充資訊

1970
CS

(FWP)

等價

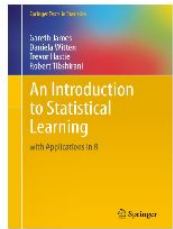
1992 1991

GPU

GPT3

1980

統計學
SL



1990

SVM

2006

DL
deep

2015

CNN

→ X

LSTM

2021

Transformer

2023

Linear Transformer



~~attention~~

~~OCN~~

Histo h